Introduzione

Description	Un'introduzione completa alla programmazione Python

La sintassi di Python è simile a quella del C tranne che per le sequenti semplificazioni

- Non è richiesto il `;` alla fine di ogni linea
- invece delle `{ }` i blocchi sono evidenziati mediante indentazione (attenti all'editor) e con il carattere `:`
- Le parentesi `( )` nei test degli `if` / `while` non sono necessarie
- `/` vs `//` per distinguere tra divisione e divisione intera. Per i commenti si usa il carattere `#`
- non ci sono `++` e `-`
- operatori logici si chiamano con il loro nome: `and`, `or`, `not` etc
- Invece di `true` / `false` / `NULL` ci sono `True` / `False` / `None`

Python è un linguaggio interpretato non compilato: molto più lento del C (fino a 100 volte); gli errori vengono segnalati a runtime con messaggi solitamente comprensibili.

Altra differenza sostanziale è che le variabili non hanno tipo e non vanno dichiarate.

Le stringhe sono immutabili e facili da usare. Non c'è il tipo carattere: un carattere è una stringa di lunghezza uno.

Gli interi hanno precisione illimitata: ci pensa l'interprete ad usare un numero adeguato di byte per rappresentare tutte le cifre (fate la prova con il fattoriale)

Strutture di controllo e iterazione

- if/else/elif
- while/continue/break, esiste un comando else per i cicli, non esiste do

- for è basato sul concetto di range, simile al for-each di Java

Definizione e invocazione di funzioni

Esempio di funzione che dati due interi restituisce la somma e la stringa contenente la loro concatenazione. Non è una funzione molto sensata: serve per far vedere che è possibile restituire più di un oggetto anche di tipo diverso.

```
# definizione
def fun(a,b):
    return a+b,str(a) +str(b)

# invocazione funzione e stampa risultato
s,p = fun(3,5)
print(s,p)
```

Costruzione stampa e modifica di una lista

```
lista = [1,2,3,"ciao"]
print(lista[2:]) # estrae gli elementi da lista[2] alla fine
lista[2] = 5
lista = lista + [34,40]
lista.append(35)
print(lista)
```

Esempi di iterazione su una lista usando il ciclo for

```
parole = ["casa","mare","sole"]
for x in parole:
    print(x.upper())
    x = "monti"
print(parole)
for i in range(len(parole)):
    print(parole[i].upper())
```

```
    i = 7
print(parole)
```

Nota: la parola chiave `in` si usa nei cicli for e per testare l'appartenenza. Verificatelo eseguendo lea seguenti istruzioni:

```
print(4 in [1,2,3])
print(4 in [2,4,8])
```

Dizionari

Sono equivalenti alle `map` di Java ma più semplici da usare

```
d = {"casa":3, "sole":4}
d["mare"] = 5
d["casa"] = 10
print(d)
for c in d:
    print("Alla chiave", c, "è associato il valore", d[c])
```

Docstring

Una stringa delimitata da tre virgolette `"""` nella riga immediatamente successiva alla definizione di una funzione di chiama *docstring* e forma un commento che l'interprete associa alla funzione. Viene mostrata con il comando *help* e da alcuni editor specifici per python. Ad esempio

```
def media(a,b):
  """Calcola la media artimetica"""
  return (a+b)/2

help(media)
Help on function media in module __main__:
```

```
media(a, b)
    Calcola la media aritmetica
```

Esecuzione di programmi

Inizialmente eseguiremo i programmi chiamando le funzioni all'interno dell'interprete python (prompt `>>>`). L'interprete ha il vantaggio che permette di testare le singole istruzioni e di vedere il contenuto delle variabili.

Successivamente eseguiremo i programmi python dalla linea di comando del terminale (in questo caso vengono chiamati `script`). Per poterli eseguire dalla linea di comando è necessario che il file contenente il sorgente inizi con la linea

```
#!/usr/bin/env python3
```

e che il file venga reso eseguibile con il comando `chmod(1)`

Namespace

Un namespace è un mapping da nomi a oggetti
implementato come un dizionario che dato un nome (variabile, metodo), Python utilizza il dizionario per reperire l'oggetto associato a quel nome

Un namespace è fondamentale per evitare conflitti di nomi e gestire la visibilità e la portata delle variabili all'interno del programma.

Ci sono diversi tipi di namespace in Python, tra cui i principali sono:

1. **Namespace Locale (Local Namespace):**
    - Il namespace locale è specifico di una funzione o di un blocco di codice. Le variabili definite all'interno di una funzione esistono solo in quel contesto e non sono visibili all'esterno.

```
pythonCopy code
def my_function():
    local_variable = 10
```

2. **Namespace Globale (Global Namespace):**
    - Il namespace globale è associato all'intero modulo o script. Le variabili definite al di fuori di tutte le funzioni sono globali e possono essere accedute da qualsiasi parte del modulo.

```
pythonCopy code
global_variable = 20
```

3. **Namespace Incorporato (Built-in Namespace):**
    - Il namespace incorporato contiene tutte le funzioni e i nomi incorporati di Python, come le funzioni built-in come `print()`, `len()`, ecc.
4. **Namespace di Modulo (Module Namespace):**
    - Ogni modulo Python ha il proprio namespace, che funge da namespace globale per le variabili all'interno del modulo. Le variabili definite nel modulo sono accessibili da altre parti dello stesso modulo.

```
pythonCopy code
# module_example.py
module_variable = 30
```

5. **Namespace Incorporato e Locale (Built-in and Local Namespace):**
    - Quando una funzione è eseguita, crea un nuovo namespace locale, ma ha ancora accesso al namespace incorporato di Python. Le variabili definite all'interno della funzione sono locali, ma le funzioni built-in sono ancora accessibili.

```
pythonCopy code
def example_function():
    local_variable = 40
    print(len("Hello"))  # Accesso al namespace incorporat
o
```

La funzione `dir()` restituisce una lista di stringhe che rappresentano i nomi disponibili nel contesto specificato.

Durante l'esecuzione del programma esistono 4 namespace, ed in essi viene cercato il binding di una variabile nell'ordine seguente

- ***scope locale***: variabili locali a una funzione
- ***enclosing scope***: scope delle funzioni che racchiudono la funzione dove si trova
  la variabile, contiene variabili non locali
- ***global scope***: lo scope contenente le variabili del modulo
- ***built-in scope***: contiene nomi built-in

Variabile __name__

In Python, `__name__` è una variabile speciale che rappresenta il nome del namespace corrente. Questa variabile è utilizzata principalmente per determinare se uno script Python è eseguito come programma autonomo o se è importato come modulo in un altro script.

Quando uno script Python è eseguito, il valore della variabile `__name__` è impostato su `__main__` se lo script è il programma principale che viene eseguito.

Se, invece, lo script è importato come modulo in un altro script, il valore di `__name__` sarà il nome del modulo stesso.

```
# Esempio di script chiamato my_script.py

def main():
    print("Questo è il programma principale.")

if __name__ == "__main__":
    # Questa parte del codice verrà eseguita solo se lo sc
```

```
ript è il programma principale
    main()
```

In questo esempio, la funzione `main()` verrà chiamata solo se lo script `my_script.py` viene eseguito direttamente. Se, invece, lo script viene importato in un altro modulo, il blocco `if __name__ == "__main__":` verrà ignorato.

Questo modello è spesso utilizzato per consentire la riutilizzabilità dei moduli Python. Quando si scrive uno script, è possibile includere codice che deve essere eseguito solo quando lo script è il programma principale, ma non quando è importato come modulo.